

Functions

A function is like a **kitchen appliance**: input ingredients → appliance → output dish.

Functions make code reusable, just like you don't bake a cake from scratch every time.

Types:

- **Named functions:** `function greet() {}`
- **Anonymous functions:** `const greet = function() {}`
- **Arrow functions:** `const greet = () => {}`

// Named function

```
function add(a,b){  
  return a+b;  
}
```

// Anonymous function

```
const multiply = function(a,b){  
  return a*b;  
}
```

// Arrow function

```
const subtract = (a,b) => a-b;
```

Difference between Function declaration expression and arrow functions.

Feature	Function Declaration	Function Expression	Arrow Function
Hoisting	☑ Fully hoisted	✗ Not hoisted	✗ Not hoisted
Syntax	Verbose	Standard	Concise
this binding	Own this	Own this	Lexical this (inherits)
Can be constructor	☑ Yes	☑ Yes	✗ No
arguments object	☑ Available	☑ Available	✗ Not available
Best Use	Core logic	Dynamic usage	Callbacks / short funcs

When to Use What (Real-World Rule)

- ✓ Use **Function Declaration** → when defining main program logic.
- ✓ Use **Function Expression** → when functions are values (callbacks, conditionally defined).
- ✓ Use **Arrow Function** → when you want short syntax and lexical this.

Function declarations are hoisted and create their own this, function expressions are not hoisted and are treated as values, while arrow functions provide concise syntax and inherit this from their surrounding scope, making them ideal for callbacks and functional patterns.